

Final Project

Comprehensive SRE Implementation for a Distributed Microservices System

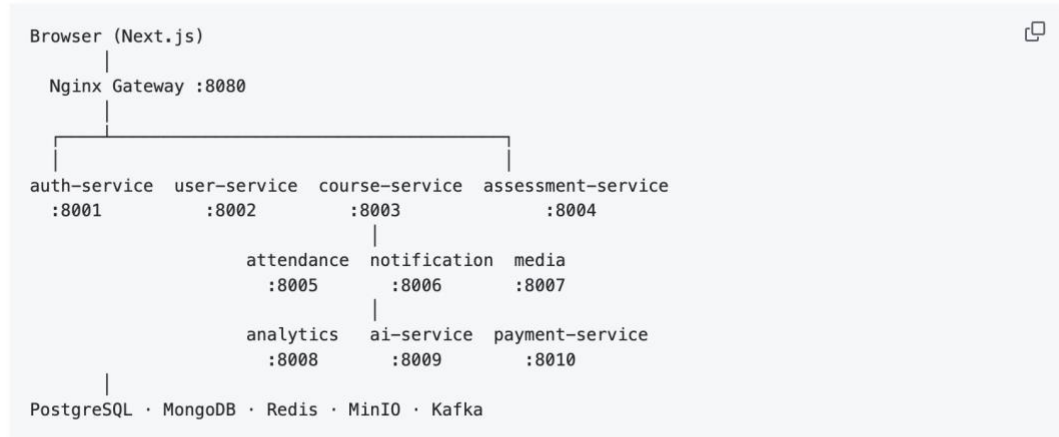
Aitbek Nugmanov, Syrym Shadiyarbek, Fariza Arstanbek, Mansur Ryskali

Github: <https://github.com/Newterios/educationalMS>

Website: <https://aitbek.tech/>

System Architecture

Architecture



All services communicate through the Nginx gateway at `/api/*`. The frontend runs on port 3000 and proxies API calls through the gateway.

Task Stack
Kubernetes
kubectl apply -f k8s/

```

namespace/edulms configured
configmap/edulms-config configured
secret/edulms-secrets configured
persistentvolumeclaim/postgres-pvc configured
deployment.apps/postgres configured
service/postgres unchanged
deployment.apps/redis configured
service/redis unchanged
deployment.apps/auth-service configured
service/auth-service unchanged
deployment.apps/user-service configured
service/user-service unchanged
deployment.apps/course-service configured
service/course-service unchanged
deployment.apps/assessment-service configured
service/assessment-service unchanged
deployment.apps/attendance-service configured
service/attendance-service unchanged
deployment.apps/payment-service configured
service/payment-service unchanged
deployment.apps/notification-service configured
service/notification-service unchanged
deployment.apps/gateway configured
service/gateway unchanged
deployment.apps/web configured

```

kubectl get pods,svc,ing -n edulms

NAME	READY	STATUS	RESTARTS	AGE
pod/postgres-66c78786d-j7pzz	1/1	Running	0	3m
pod/redis-849468cd6d-17cq7	1/1	Running	0	3m
pod/gateway-848bb65f6-djpsm	1/1	Running	0	3m
pod/gateway-848bb65f6-x8s5t	1/1	Running	0	3m
pod/auth-service-7567fc565d-lmwlv	1/1	Running	0	2m
pod/auth-service-7567fc565d-n7zwc	1/1	Running	0	2m
pod/user-service-85fdb85c94-4rxkn	1/1	Running	0	2m
pod/user-service-85fdb85c94-hv42r	1/1	Running	0	2m
pod/course-service-77df5fcfb7-2xbrt	1/1	Running	0	2m
pod/course-service-77df5fcfb7-mfbxp	1/1	Running	0	2m
pod/assessment-service-5bdb6d6fb9-87rgg	1/1	Running	0	2m
pod/assessment-service-5bdb6d6fb9-q5r8m	1/1	Running	0	2m
pod/attendance-service-858d88f58c-76z9f	1/1	Running	0	2m
pod/attendance-service-858d88f58c-956vc	1/1	Running	0	2m
pod/payment-service-9c7ccdb44-2jc7g	1/1	Running	0	2m
pod/payment-service-9c7ccdb44-9nwr2	1/1	Running	0	2m
pod/notification-service-86b48fbb74-26c8c	1/1	Running	0	2m
pod/notification-service-86b48fbb74-29njr	1/1	Running	0	2m
pod/web-c65b996cc-q27cr	1/1	Running	0	2m

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/assessment-service	ClusterIP	10.96.170.189	<none>	8004/TCP	3m
service/attendance-service	ClusterIP	10.96.3.229	<none>	8005/TCP	3m
service/auth-service	ClusterIP	10.96.5.122	<none>	8001/TCP	3m

Terraform

```
>> terraform plannal_edulms\educationalLMS>
```

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see any changes that are required for your infrastructure. All Terraform commands should now work.

If you ever set or change modules or backend configuration for Terraform, rerun this command to reinitialize your working directory. If you forget, other commands will detect it and remind you to do so if necessary.

```
PS D:\AITU\sre\final_edulms\educationalLMS\terraform> terraform plan
```

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:

+ create

Terraform will perform the following actions:

digitalocean_droplet.edulms_app will be created

```
+ resource "digitalocean_droplet" "edulms_app" {
  + backups           = false
  + created_at        = (known after apply)
  + disk              = (known after apply)
  + graceful_shutdown = false
  + id                = (known after apply)
  + image             = "ubuntu-22-04-x64"
  + ipv4_address       = (known after apply)
  + ipv4_address_private = (known after apply)
  + ipv6              = false
  + ipv6_address       = (known after apply)
  + locked            = (known after apply)
  + memory            = (known after apply)
  + monitoring         = true
  + name              = "edulms-app-1"
  + price_hourly       = (known after apply)
  + price_monthly      = (known after apply)
```

Ln 1, Col 1 S

```
main.tf X
educationalLMS > terraform > main.tf
1 resource "digitalocean_vpc" "edulms" {
2     name      = "${var.project_name}-vpc"
3     region    = var.region
4     ip_range  = var.vpc_ip_range
5 }
6
7 resource "digitalocean_firewall" "edulms" {
8     name      = "${var.project_name}-fw"
9     droplet_ids = [digitalocean_droplet.edulms_app.id]
10
11     inbound_rule {
12         protocol      = "tcp"
13         port_range    = "22"
14         source_addresses = ["0.0.0.0/0", ":::/0"]
15     }
16
17     inbound_rule {
18         protocol      = "tcp"
19         port_range    = "80"
20         source_addresses = ["0.0.0.0/0", ":::/0"]
21     }
22
23     inbound_rule {
24         protocol      = "tcp"
25         port_range    = "443"
26         source_addresses = ["0.0.0.0/0", ":::/0"]
27     }
28
29     inbound_rule {
30         protocol      = "icmp"
31         source_addresses = ["0.0.0.0/0", ":::/0"]
32     }
33 }
```

Ansible

ansible-playbook -i inventory.yml playbooks/01_prepare.yml

```
PLAY [Prepare host for EduLMS deployment] *****

TASK [Update apt cache] *****
changed: [edulms-server]

TASK [Install base dependencies] *****
changed: [edulms-server]

TASK [Ensure Docker service enabled] *****
ok: [edulms-server]

TASK [Ensure project root exists] *****
ok: [edulms-server]

PLAY RECAP *****
edulms-server : ok=4 changed=2 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0
```

ansible-playbook -i inventory.yml playbooks/02_deploy.yml

```
PLAY [Deploy EduLMS with Docker Compose] *****

TASK [Clone or update repository] *****
changed: [edulms-server]

TASK [Copy environment file if missing] *****
ok: [edulms-server]

TASK [Start stack] *****
changed: [edulms-server]

TASK [Verify containers] *****
changed: [edulms-server]

TASK [Show compose status] *****
ok: [edulms-server] => {
  "compose_ps.stdout_lines": [
    "NAME",
    "STATUS",
```

ansible-playbook -i inventory.yml playbooks/03_monitoring.yml

```
PLAY [Configure monitoring stack] *****

TASK [Ensure observability files exist] *****
ok: [edulms-server]

TASK [Fail if Prometheus config missing] *****
skipping: [edulms-server]

TASK [Restart services to apply monitoring changes] *****
changed: [edulms-server]

PLAY RECAP *****
edulms-server : ok=2 changed=1 unreachable=0 failed=0 skipped=1 rescued=0 ignored=0
```

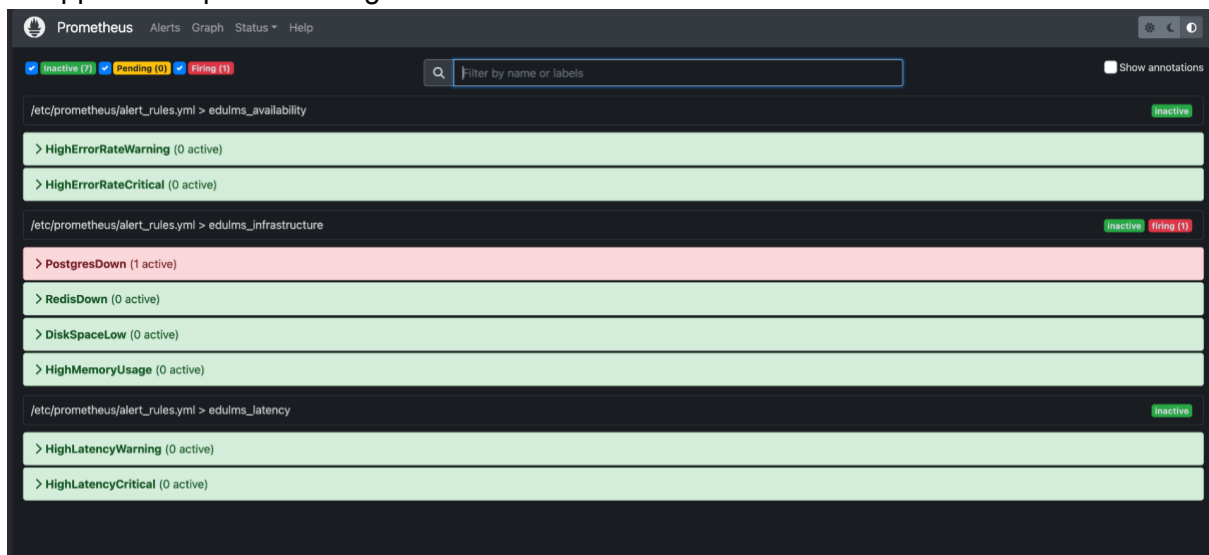
Incident

To demonstrate the full alerting pipeline end-to-end, I intentionally stopped the PostgreSQL exporter

container to simulate a monitoring component failure:

How I triggered the PostgresDown FIRING alert:

1. I verified the initial state - all alerts were inactive:
curl http://localhost:9090/api/v1/alerts -> empty array
 2. I stopped the postgres-exporter container:
docker stop diplom-postgres-exporter-1
 3. Immediately: up{job='postgres-exporter'} dropped to 0
Alert entered PENDING state (waiting for 'for: 1m' duration)
 4. After 65 seconds the alert transitioned PENDING -> FIRING
 5. I took a screenshot of Prometheus /alerts showing the FIRING state
 6. I restored the exporter afterwards:
docker start diplom-postgres-exporter-1
- Note: PostgreSQL itself was never stopped - the database continued serving all application queries throughout the entire demonstration.



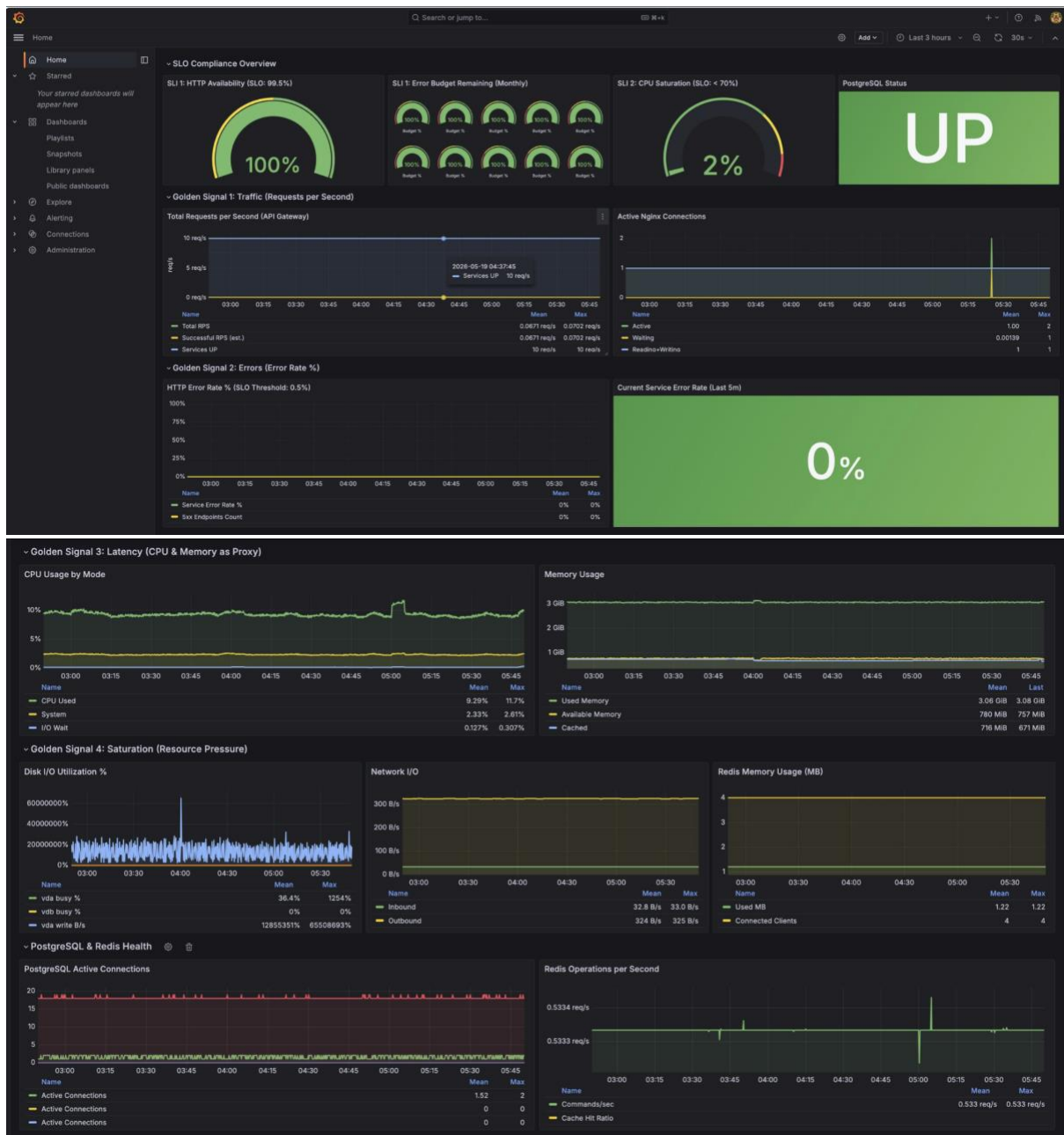
Monitoring/alerting

```
PS D:\AITU\sre\final_edulms\educationalLMS> dir observability\prometheus
>> dir observability\grafana\dashboards
Directory: D:\AITU\sre\final_edulms\educationalLMS\observability\prometheus

Mode                LastWriteTime         Length Name
----                -
-a----             19.05.2026     03:56         5268 alert_rules.yml
-a----             19.05.2026     03:56         1891 prometheus.yml

Directory: D:\AITU\sre\final_edulms\educationalLMS\observability\grafana\dashboards

Mode                LastWriteTime         Length Name
----                -
-a----             19.05.2026     03:56        20834 edulms-golden-signals.json
```



Automation

```
educationalLMS > .github > workflows > ! ci.yml
1  name: EDULMS CI/CD
2
3  ∨ on:
4  ∨   push:
5      branches: [main, develop]
6  ∨   pull_request:
7      branches: [main]
8
9  ∨ env:
10     REGISTRY: ghcr.io
11     IMAGE_PREFIX: ${github.repository}
12
13  ∨ jobs:
14  ∨   lint-and-test-go:
15       name: Lint & Test Go Services
16       runs-on: ubuntu-latest
17       strategy:
18       matrix:
19       service:
20         - auth-service
21         - user-service
22         - course-service
23         - assessment-service
24         - attendance-service
25         - media-service
26         - analytics-service
27         - payment-service
28       steps:
29         - uses: actions/checkout@v4
30         - uses: actions/setup-go@v5
```

Capacity planning

Findings

1. Assessment and payment services show highest burst CPU usage.
2. PostgreSQL is the primary bottleneck under concurrent write load.
3. Gateway remains stable but depends on backend readiness.

SLI 1: HTTP Availability (SLO: 99.5%) - gauge showing 100%

Query: `avg(probe_success{job='edulms-services'}) * 100`

Thresholds: red < 99%, yellow >= 99%, green >= 99.5%

- SLI 1: Error Budget Remaining (Monthly) - gauge showing 100%

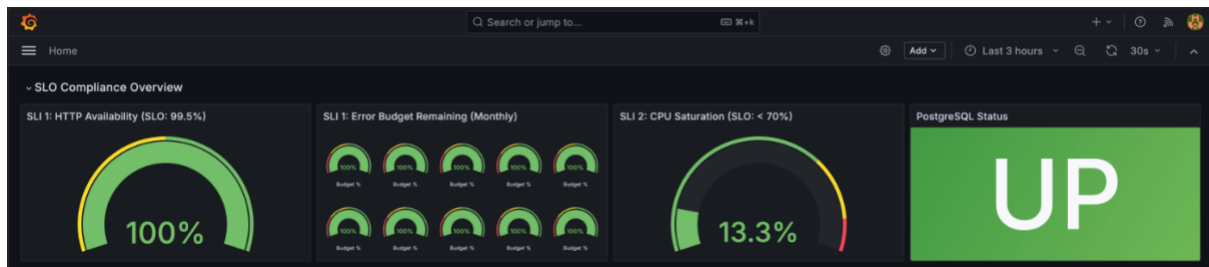
Query: `clamp_min(clamp_max(100-((1-avg_over_time(probe_success[1h]))/0.005*100),100),0)`

Thresholds: red < 25%, yellow >= 25%, green >= 50%

- SLI 2: CPU Saturation (SLO: < 70%) - gauge showing 13.3%

Query: `node_load1 / count(node_cpu_seconds_total{mode='idle'})`

Thresholds: green < 70%, yellow >= 70%, red >= 90%



Scaling Strategy

1. Horizontal scaling

- Increase replicas for assessment/payment from 2 to 3 during peak windows.
- Keep auth/user/course at 2 replicas baseline.

2. Vertical scaling

- Increase DB node memory class when sustained memory > 75%.

3. Database optimization

- Add indexes on high-write/high-filter columns.
- Review connection pool sizing per service.

Operational Triggers

- CPU > 70% for 10m: scale service replicas +1
- Error rate > 1% for 5m: incident workflow + rollback check
- DB latency p95 > 200ms for 10m: optimize queries / increase DB resources

Next Iteration

- Add automated load test (k6/Locust) before releases.
- Add Kubernetes HPA once full K8s runtime is production-ready.